

Guía rápida de Kotlin

Declarar variables y controlar nulos · Ejemplos basados en AppPizzeria

1. Declarar variables

val: valor fijo (no se puede cambiar)

```
val nombreApp = "AppPizzeria"
nombreApp = "Otra"           // ERROR: un val no se puede reasignar
```

var: variable (se puede cambiar)

```
var contador = 0
contador = contador + 1    // OK
```

Usa **val** siempre que puedas, y **var** solo cuando el valor tenga que cambiar.

Tipo automático o explícito

```
val precio = 8.5           // Kotlin deduce que es Double
val pizzas: Int = 3        // tipo indicado a mano
val nombre: String = "José"
```

Tipos más comunes

Tipo	Ejemplo
<code>String</code>	<code>"Hola"</code>
<code>Int</code>	<code>5</code>
<code>Double</code>	<code>8.5</code>
<code>Boolean</code>	<code>true / false</code>
<code>List<String></code>	<code>listOf("a", "b")</code>

En `findViewById<TextView>(...)` usas **val** aunque luego cambies `tvRumor.text`. Lo que no cambia es la variable, que siempre apunta al mismo `TextView`; lo que cambia es el contenido de ese `TextView`.

2. Nulos (null)

null significa “no hay valor”. Kotlin no deja que una variable sea **null** salvo que lo indiques con **?**.

```
val a: String = null        // ERROR
val b: String? = null       // OK: el ? permite que sea null
```

Dónde te vas a encontrar nulos en Android

```
intent.extras                // Bundle? (puede no haber datos)
datos?.getString("nombre")   // String? (puede no existir la clave)
etnPizzas.text.toString().toIntOrNull() // Int? (puede no ser un número)
```

3. Herramientas para controlar nulos

?.: llamada segura

Si es `null`, no hace nada y devuelve `null`, en vez de cerrar la app.

```
val datos = intent.extras
val nombre = datos?.getString("nombre") // si datos es null → nombre es null
```

?:: valor por defecto (operador Elvis)

“Si es `null`, usa esto otro.”

```
val nombre = datos?.getString("nombre") ?: "Sin nombre"
val pizzas = etnPizzas.text.toString().toIntOrNull() ?: 0
```

Así `nombre` es un `String` normal (no `String?`), porque nunca podrá ser `null`.

if (x == null): comprobar y cortar

```
val numPizzas = etnPizzas.text.toString().toIntOrNull()

if (numPizzas == null) {
    Toast.makeText(this, "Número no válido", Toast.LENGTH_LONG).show()
    return@setOnClickListener
}

// Aquí Kotlin ya sabe que numPizzas NO es null (smart cast)
val total = numPizzas * 8
```

?.let { }: hacer algo solo si no es null

```
datos?.getString("nombre")?.let {
    tvNombre.text = it // "it" es el valor, ya sin null
}
```

!!: forzar (evítalo)

```
val nombre = datos!!.getString("nombre")!! // si algo es null, la app se cierra
```

Le dice a Kotlin “confía en mí, no es null”. Si te equivocas, la app se cierra con un `NullPointerException`. Usa mejor `?.` o `?::`.

4. Comprobar textos vacíos

Función	Devuelve true si...
<code>isEmpty()</code>	el texto es ""
<code>isBlank()</code>	el texto es "" o solo espacios " "
<code>isNullOrEmpty()</code>	es null o ""
<code>isNullOrBlank()</code>	es null, "" o solo espacios

```
if (etNombre.text.toString().isBlank()) { ... } // EditText (nunca es null)
if (nombre.isNullOrBlank()) { ... }           // String? recogido de un Intent
```

5. Convertir texto a número de forma segura

Función	Si el texto no es un número...
<code>toInt()</code>	la app se cierra
<code>toIntOrNull()</code>	devuelve null (seguro)
<code>toDouble()</code>	la app se cierra
<code>toDoubleOrNull()</code>	devuelve null (seguro)

Usa siempre la versión `OrNull` con lo que escriba el usuario.

6. Resumen

```
val x = 5           // fijo
var y = 5           // cambiable
val z: String? = null // puede ser null

z?.length           // llamada segura → null si z es null
z ?: "por defecto"   // valor alternativo si es null
if (z != null) { z.length } // dentro del if, z ya no es null
z?.let { println(it) } // solo se ejecuta si no es null
z!!.length           // EVITAR: se cierra la app si z es null
```